

1.Remove White Space

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    ifstream file("lex.cpp");
    if (!file.is_open()) {
        cout << "File cannot open" << endl;
        return 0;
    }
    char ch;
    while(file.get(ch)){
        if(!isspace(ch))
            cout<<ch;
    }

    file.clear();
    file.seekg(0);

    while(file.get(ch)){
        if(isdigit(ch)){
            while(isdigit(ch)){
                cout<<ch;
                if(!file.get(ch))
                    break;
            }
            cout<<endl;
        }
    }
    file.close();
}
```

2.White Space & Collect Number as Token

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    ifstream file("lex.cpp");
    if(!file.is_open()){
        cout<<"file is not open";
        return 0;
    }

    char ch;
    while(file.get(ch)){
        if(!isspace(ch)){
            cout<<ch;
        }
    }
    cout<<"\n \n";

    file.clear();
    file.seekg(0);

    while(file.get(ch)){
        if(isdigit(ch)){
            string num="";
            while(isdigit(ch)){
                num+=ch;
                if(!file.get(ch)) break;
            }
            cout<<"Num \t\t"<<num<<endl;
        }
    }
    file.close();
}
```

3.variable

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    ifstream file("lex.cpp");
    if(!file.is_open()){
        cout<<"File cannot open"<<endl;
        return 0;
    }

    string word;

    string keyword[]={
        "int","float","double","if","else","for",
        "while","return","void","break","continue",
        "includebitsstdch","using","namespace","std","main"
    };

    while(file>>word){
        string var="";
        for(char ch:word){
            if(isalnum(ch) || ch=='_') var+=ch;
        }

        if(var=="") continue;

        bool isKeyword=false;

        for(string k:keyword){
            if(var==k){
                isKeyword=true;
                break;
            }
        }

        if(!isKeyword && (isalpha(var[0]) || var[0] =='_')){
            cout<<var<<endl;
        }
    }
    file.close();
}
```

4.keyword

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    ifstream file("lex.cpp");
    if(!file.is_open()){
        cout<<"this file is not be open";
        return 0;
    }

    string word;

    string keyword[]={
        "int","float"
    };

    while(file>>word){
        string key="";
        for(char ch:word){
            if(isalnum(ch) || ch=='_') key+=ch;
        }
        for(string k:keyword){
            if(k==key){
                cout<<k<<<endl;
            }
        }
    }
    file.close();
}
```

5. Infix to Postfix

```
#include<bits/stdc++.h>
using namespace std;
int priority(char op){
    if(op=='+' || op=='-') return 1;
    if(op=='*' || op=='/') return 2;
    return 0;
}
int main(){
    string infix;cin>>infix;
    stack<char>st;

    for(char ch:infix){
        if(isalnum(ch))cout<<ch;
        else if(ch=='(') st.push(ch);
        else if(ch==')'){
            while(!st.empty() && st.top()!='('){
                cout<<st.top();
                st.pop();
            }
            if(!st.empty()) st.pop();
        }
        else if(ch==';'){
            while(!st.empty()){
                cout<<st.top();
                st.pop();
            }
            break;
        }
        else{
            while(!st.empty() && st.top() != '(' &&
                priority(st.top()) >= priority(ch)){
                cout<<st.top();
                st.pop();
            }
            st.push(ch);
        }
    }
    cout<<endl;
}
```

6) Left shifting

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    string str;cin>>str;
    int pos=str.find('|');

    string p1=str.substr(0,pos);
    string p2=str.substr(pos+1);

    string common="";
    int i=0;

    while(i<p1.size() && i<p2.size() && p1[i]==p2[i]){
        common+=p1[i];
        i++;
    }
    if(common=="") cout<<"Fact ! pos"<<endl;
    else{
        cout<<"A-> "<<common<<"A'"<<endl;
        cout<<"A' -> ";

        if(p1.substr(i)== "") cout<<"#";
        else cout<<p1.substr(i);
        cout<<"|";
        if(p2.substr(i)== "") cout<<"#";
        else cout<<p2.substr(i);
    }
}
```

7)Grammer Rules

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    string str;

    cout << "Grammar:\n";
    cout << "S -> aS\n";
    cout << "S -> Sb\n";
    cout << "S -> ab\n\n";

    cout << "Enter the string: ";
    cin >> str;

    int len = str.size();
    int flag = 0;
    int a = -1, b = len;
    int i;

    // Find the last 'a'
    for (i = 0; i < len; i++)
    {
        if (str[i] != 'a')
        {
            a = i - 1;
            break;
        }
    }

    // If the whole string is 'a'
    if (i == len)
        a = len - 1;

    // Find the first 'b'
    for (i = len - 1; i >= 0; i--)
    {
        if (str[i] != 'b')
        {
            b = i + 1;
            break;
        }
    }
}
```

```

// If the whole string is 'b'
if (i == -1)
    b = 0;

// Check the grammar
if (a >= 0 && b < len && a + 1 == b)
    flag = 1;

if (flag)
    cout << "\n" << str << " is Accepted.\n";
else
    cout << "\n" << str << " is Rejected.\n";

return 0;
}

```

6)Left factoring

1 code learn

এই প্রোগ্রামটি একটি **Lexical Analysis**-এর ছোট উদাহরণ। এটি `lex.cpp` ফাইল থেকে ডাটা পড়ে দুইটি কাজ করে:

1. সব whitespace (space, tab, newline) বাদ দিয়ে কোড প্রিন্ট করে।
2. কোডের মধ্যে থাকা সব সংখ্যা (numbers) আলাদা করে বের করে প্রিন্ট করে।

এখন লাইন ধরে বুঝি।

Header File

```

#include<bits/stdc++.h>
using namespace std;

```

- `#include<bits/stdc++.h>` → প্রায় সব C++ লাইব্রেরি একসাথে include করে।
 - `using namespace std;` → `std::` না লিখেও `cout`, `ifstream` ইত্যাদি ব্যবহার করা যায়।
-

Main Function


```
int main(){
```

এখান থেকেই প্রোগ্রাম শুরু।

File Open

```
ifstream file("lex.cpp");
```

ifstream = Input File Stream

এটি lex.cpp নামের ফাইলটি **Read Mode**-এ খুলে।

ধরুন lex.cpp-তে আছে

```
int a = 10;  
float b = 20;
```

File Open হয়েছে কিনা

```
if(!file.is_open()){  
    cout<<"No file";  
    return 0;  
}
```

যদি ফাইল না পাওয়া যায় তাহলে

```
No file
```

দেখাবে এবং প্রোগ্রাম শেষ হবে।

Character Variable

```
char ch;
```

একটি করে Character রাখার জন্য।

প্রথম while Loop

```
while(file.get(ch)){
```

`file.get(ch)` একবারে **একটি Character** পড়ে।

যেমন

```
int a=10;
```

পড়বে

```
i  
n  
t  
(space)  
a  
=  
1  
0  
;
```

একটি একটি করে।

Space বাদ দেওয়া

```
if(!isspace(ch)){  
    cout<<ch;  
}
```

`isspace()` দেখে Character Space কিনা।

যদি হয়

```
' '  
'\n'  
'\t'
```

তাহলে True।

`!isspace(ch)` মানে

Space না হলে প্রিন্ট করো।

Example

ফাইলে

```
int a = 10;
```

আউটপুট

```
inta=10;
```

সব Space চলে যাবে।

এবার সমস্যা

এখন File একবার শেষ পর্যন্ত পড়ে ফেলেছে।

Pointer এখন EOF-এ (End Of File)।

তাই আবার পড়তে গেলে কিছুই পাবে না।

তাই

```
file.clear();
```

EOF Flag Remove করে।

Pointer আবার শুরুতে নেয়

```
file.seekg(0);
```

seekg(0) মানে

Pointer

↓

```
int a=10;
```

আবার প্রথম Character-এ চলে যায়।

দ্বিতীয় while Loop

```
while(file.get(ch)){
```

আবার Character পড়া শুরু।

Digit Check

```
if(isdigit(ch)){
```

যদি Character

0-9

হয় তাহলে ভিতরে যাবে।

ভিতরের while

```
while(isdigit(ch)){
```

যতক্ষণ সংখ্যা থাকবে ততক্ষণ চলবে।

ধরুন

12345

তাহলে

1
12
123
1234
12345

এভাবে Character পড়বে।

Print

```
cout<<ch;
```

সংখ্যাগুলো প্রিন্ট করবে।

পরের Character পড়া

```
if(!file.get(ch)){  
    break;  
}
```

আরও Character আছে কিনা দেখে।

না থাকলে বের হয়ে যাবে।

Number শেষ হলে

```
cout<<endl;
```

পরের লাইনে যাবে।

Example

ধরুন

```
int a=100;  
int b=250;  
float c=3;
```

প্রথম অংশ

```
inta=100;intb=250;floatc=3;
```

দ্বিতীয় অংশ

```
100
250
3
```

শেষে

```
file.close();
```

ফাইল বন্ধ করে।

`file.clear()` **কেন দরকার?**

কারণ প্রথম `while` শেষ হওয়ার পর File Pointer এমন অবস্থায় থাকে:

```
i n t a = 1 0 ;
                ↑
                EOF
```

EOF-তে থাকলে আর কিছু পড়া যায় না।

`file.clear();` → EOF Flag মুছে দেয়।

`file.seekg(0);` → Pointer আবার শুরুতে নিয়ে যায়।

Token কী?

Token হলো Source Code-এর সবচেয়ে ছোট অর্থপূর্ণ (meaningful) অংশ, যেটা Lexical Analyzer চিনতে পারে।

ধরুন এই C কোডটি আছে:

```
int a = 100 + 20;
```

এখানে Token গুলো হলো:

Source Token Type

int Keyword

Source Token Type

a	Identifier
=	Operator
100	Number
+	Operator
20	Number
;	Separator

অর্থাৎ,

```
int a = 100 + 20;
```

Lexical Analyzer এটাকে ভেঙে দেখে:

```
[int] [a] [=] [100] [+] [20] [;]
```

এগুলোর প্রতিটিই একটি **Token**।

Token Value (Attribute) কী?

Token-এর সাথে যে তথ্য থাকে তাকে **Attribute** বলে।

উদাহরণ:

```
int age = 50;
```

Token	Attribute
-------	-----------

int	Keyword
-----	---------

age	Identifier name
-----	-----------------

=	Assignment operator
---	---------------------

50	Value = 50
----	------------

এখানে

```
Token = Number  
Attribute = 50
```

অর্থাৎ

Number → 50

এটাই প্রশ্নে বলা হয়েছে:

Collect numbers as token and display the token value as attribute.

এই কোডে Token কোথায়?

এই অংশে:

```
if(isdigit(ch)){
```

যখন কোনো সংখ্যা পাওয়া যায়,

যেমন

100

তখন প্রোগ্রাম বুঝে

এটা Number Token

এরপর

```
string num;
```

একটা খালি String তৈরি হয়।

```
while(isdigit(ch)){
```

যতক্ষণ সংখ্যা থাকবে,

1
10
100

সব String-এ যোগ হবে।

Input: (A + B)*C-D;

পুরো ট্রেস টেবিল

Character	Stack	Output
(	(	
A	(	A
+	(+	A
B	(+	AB
)	Empty	AB+
*		AB+

Character	Stack	Output
C	*	AB+C
-	-	AB+C*
D	-	AB+C*D
;	Empty	AB+C*D-

6) Left Factoring

এই Program কী করে?

এটি শুধু দুটি Production নিয়ে কাজ করে।

যেমন Input

abc | abd

মানে

$A \rightarrow abc \mid abd$

Program বের করবে

$A \rightarrow abA'$

$A' \rightarrow c \mid d$

প্রথমে Concept বুঝি

Left Factoring কী?

যখন দুইটি Production একই Prefix দিয়ে শুরু হয়, তখন সেই Common Prefix বাইরে বের করে আনা হয়।

এতে Parser-এর জন্য Grammar সহজ হয়।

Example 1

Grammar

$$A \rightarrow abX \mid abY$$

দেখো দুইটিই

ab

দিয়ে শুরু হয়েছে।

এটাই Common Prefix।

তাই Left Factoring করলে

$$\begin{aligned} A &\rightarrow abA' \\ A' &\rightarrow X \mid Y \end{aligned}$$

Example 2

Grammar

$$A \rightarrow abc \mid abd$$

Common Prefix

ab

Left Factoring

$$\begin{aligned} A &\rightarrow abA' \\ A' &\rightarrow c \mid d \end{aligned}$$

Example 3

Grammar

$$A \rightarrow \text{ifEthenSelseS} \mid \text{ifEthenS}$$

Common Prefix

ifEthenS

Left Factoring

```
A → ifEthenSA'  
A' → elseS | ε
```

এই Program কী করে?

এটি শুধু দুটি **Production** নিয়ে কাজ করে।

যেমন Input

```
abc|abd
```

মানে

```
A → abc | abd
```

Program বের করবে

```
A → abA'  
A' → c | d
```

এখন Code বুঝি

Input

```
string str;  
cin >> str;
```

ধরি Input

```
abc|abd
```

```
find('|')
```

```
int pos = str.find('|');
```

এটি | কোথায় আছে সেটা বের করে।

Input

abc|abd

Index

0 1 2 3 4 5 6
a b c | a b d

তাই

pos = 3

প্রথম Production

```
string p1 = str.substr(0,pos);
```

মানে

0 থেকে pos-1 পর্যন্ত

অর্থাৎ

abc

দ্বিতীয় Production

```
string p2 = str.substr(pos+1);
```

মানে

'|' এর পরে যা আছে

অর্থাৎ

abd

এখন

p1 = abc

p2 = abd

Common Prefix রাখার জন্য

```
string common = "";
```

প্রথমে খালি।

Index

```
int i = 0;
```

সবচেয়ে গুরুত্বপূর্ণ Loop

```
while(i < p1.size() &&  
      i < p2.size() &&  
      p1[i] == p2[i])
```

এর মানে

যতক্ষণ দুই String-এর Character একই থাকবে ততক্ষণ Loop চলবে।

Dry Run

```
p1 = abc  
p2 = abd
```

প্রথমবার

```
i = 0
```

```
p1[0] = a
```

```
p2[0] = a
```

একই

তাই

```
common += p1[i];
```

Common

a

দ্বিতীয়বার

i = 1

p1 = b

p2 = b

একই

Common

ab

তৃতীয়বার

i = 2

p1 = c

p2 = d

এক নয়।

Loop শেষ।

এখন

common = ab

i = 2

Common না থাকলে

if (common=="")

মানে

```
abc|xyz
```

Common কিছু নেই।

Output

```
Fact ! pos
```

(এখানে আসলে "Fact not possible" লেখা উচিত।)

Common থাকলে

```
cout<<"A-> "<<common<<"A'"<<endl;
```

Output

```
A -> abA'
```

এরপর

```
cout<<"A' -> ";
```

(তোমার কোডে ভুল করে A-> লেখা হয়েছে, এখানে A' -> হওয়া উচিত।)

প্রথম Production-এর বাকি অংশ

```
p1.substr(i)
```

এখন

```
i = 2
```

তাই

```
abc
```

থেকে

```
c
```


পাবে।

দ্বিতীয় Production

`p2.substr(i)`

পাবে

d

Output

$A \rightarrow abA'$
 $A' \rightarrow c \mid d$

আরেকটি Example

Input

`abcd|abef`

Common

ab

Output

$A \rightarrow abA'$
 $A' \rightarrow cd \mid ef$

যদি Input হয়

`abc|abc`

Common

abc

বাকি অংশ

"" | ""

তখন

```
if (p1.substr(i)=="")
```

সত্য হবে।

তাই

#

প্রিন্ট করবে।

Output

A → abcA'
A' → # | #

এখানে # মানে ϵ (epsilon) বা Empty String।